代码说明 • 技术设计 • 复现验证

Web 科学计算小程序技术说明

FastAPI · React · Plotly

代码：web/

功能：PSF 仿真生成 + CNN/BOBYQA 反演

启动：uvicorn web.api.main:app

版本：v1.0 • 2026-11

> *后端是 64×64 PSF 教学模型；不是完整 256²/15 星 CCD 管线。*

01 功能与架构

浏览器里的仿真/反演闭环。

代码 1-1 架构。

```
browser (React + Plotly)
   |  fetch JSON
FastAPI /api/simulate, /api/invert
   |  torch forward / CNN / NLopt
ml/phase_torch.py, ml/two_stage.py
```

表 1-1 功能/API

功能	接口	输出
参数滑杆	—	c4/c8/c9/noise
生成 5 帧 PSF	POST /api/simulate	PNG data URL
真实感 CCD	POST /api/ccd	320×320 星场 PNG
反演	POST /api/invert	truth/CNN/refined/nfev/cost
健康检查	GET /api/health	模型状态

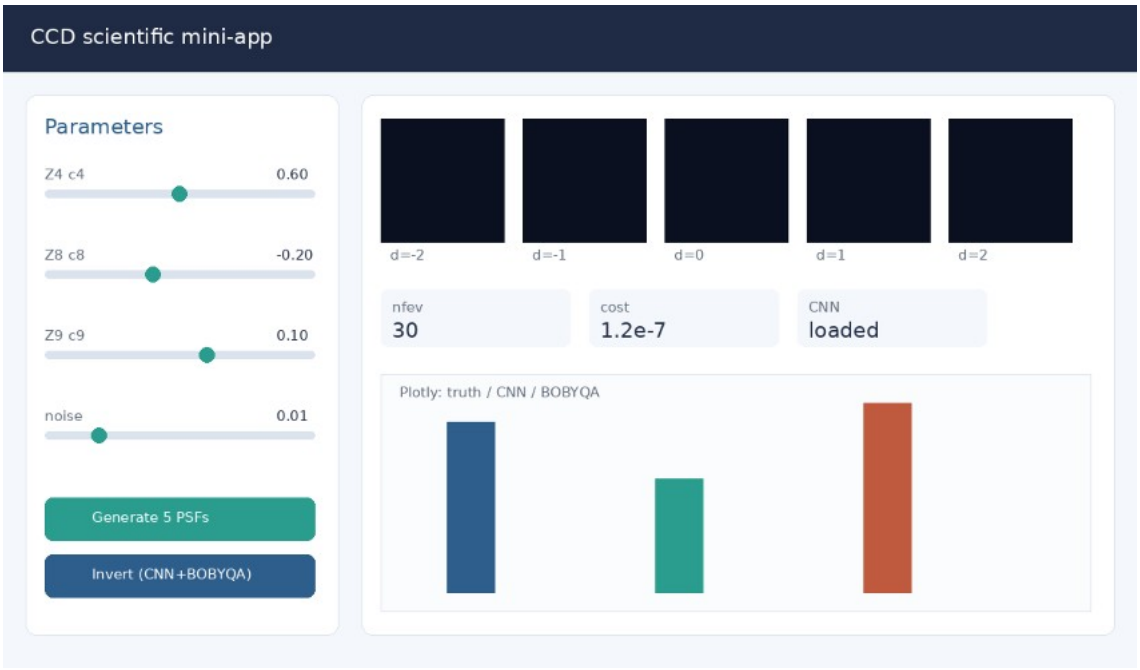


图 1-1 页面线框：左栏参数/按钮，右栏 5 帧、指标卡、Plotly 对比。

02 后端设计

接口、归一化、错误处理。

代码 2-1 反演接口。

```
@app.post("/api/invert")
def invert(req):
    stack = forward_stack(torch.tensor(truth), cfg, add_noise=True).numpy()
    cnn = model(peak_normalise(stack)) * scales
    x, nfev, cost = bobyqa(cnn, sum_normalise(stack))
    return {"truth":..., "cnn":..., "refined":..., "nfev":nfev, "cost":cost}
```

CNN 输入按帧峰值归一化，BOBYQA 目标按 sum 归一化；两者不能混。模型缺失时 `init=0` 仍可跑。接口返回 `shape`，前端才能正确显示帧数/尺寸。

表 2-1 常见故障

问题	现象	处理
反演卡住	后端 500/超时	检查 uvicorn 日志；接口 90 s 超时；返回错误文本
图表不显示	`#chart` 不存在/Plotly 未加载	React 内渲染 chart；drawChart 先判空
CNN 未加载	model_loaded=false	先训练 ml/runs/cnn.pt；零初值兜底
跨域	浏览器 CORS	同源部署；不要直接 file:// 打开

2.2 请求/响应示例

代码 2-2 反演请求/响应。

```
POST /api/invert
{"c4":0.6,"c8":-0.2,"c9":0.1,"noise":0.01}
-> {"truth":[0.6,-0.2,0.1], "cnn":[...], "refined":[...],
    "nfev":30, "cost":1.2e-7, "model_loaded":true, "shape":[5,64,64]}
```

如果 `model_loaded=false`，说明 ml/runs/cnn.pt 不存在；反演仍从零初值跑 BOBYQA，但精度/时间会变。接口返回的 `shape` 用于前端指标卡，缺少它会导致页面报错。

2.3 为什么反演会“卡住”

已定位：早期页面没有渲染 `#chart`，反演返回后 `Plotly.react('chart',...)` 对不存在的元素执行，React effect 抛错，页面看起来卡住。修复：React 树中渲染 `<div id='chart'>`；drawChart 先判空；fetch 加 90 s 超时和错误文本。

03 前端设计

> React 单页 + Plotly。

React 18 UMD + Babel Standalone（浏览器内 JSX）+ Plotly.js CDN；无 npm 构建。左栏参数/按钮，右栏 5 帧缩略图、指标卡、Plotly 柱状图。

代码 3-1　启动/自测。

```
python3 -m uvicorn web.api.main:app --reload --port 8000
# http://127.0.0.1:8000/
python3 web/test_api.py
```

实测：health ok/model true；simulate 5 帧 [5,64,64]；invert truth=[0.6,-0.2,0.1]，refined≈[0.596,-0.220,0.111]，nfev≈30；index 200。

3.2 React 状态与副作用

`sim` 保存帧/形状，`inv` 保存反演结果，`busy` 控制按钮，`err` 显示错误。首屏 useEffect 自动生成并绑定左栏按钮；第二个 useEffect 在 `inv` 变化后画 Plotly。注意 `doInv` 里先 setState，图表在 effect 里画，避免在 DOM 更新前访问 `#chart`。

3.3 部署

单机：uvicorn 直接跑；生产：FastAPI 放在 Nginx 后，任务队列处理长任务，React 打包进静态资源。当前 CDN 只适合 demo；离线部署把 React/Plotly/Babel 下载到 web/ui/vendor 并改 script src。

3.4 边界

- 64×64 PSF，不是完整 CCD 星场；
- CDN 依赖网络；离线需 vendor；
- 单用户 demo，无任务队列/数据库；
- 生产化用 FastAPI + Celery/RQ + 对象存储。

3.5 测试与观测

表 3-1 Web 验证

项	方法	当前结果
健康检查	GET /api/health	ok/model true
仿真接口	POST /api/simulate	5 帧 [5,64,64]
反演接口	POST /api/invert	nfev≈30, cost≈1e-7
页面	GET /	200
日志	uvicorn access log	可看到 500/超时

前端错误处理：fetch 90 s 超时，失败在页面显示文本；不要只留一个转圈按钮。后端错误处理：模型缺失时零初值兜底，接口返回 model_loaded=false。

3.6 后续

- 把完整 256²/15 星 CCD 管线接进页面（任务队列 + FITS 下载）；
- 加 WebSocket/SSE 显示 BOBYQA 迭代；
- 加用户输入校验/限流/日志脱敏；
- 离线部署：React/Plotly vendor 本地化，Docker 多阶段构建。

附录 A 关键源码

代码 A-1 FastAPI 反演端点。

```
@app.post("/api/invert")
def invert(req: SimRequest):
    stack = forward_stack(torch.tensor(truth), cfg, add_noise=True).numpy()
    cnn = model(peak_normalise(stack)) * scales
    x, nfev, cost = bobyqa(cnn, sum_normalise(stack))
    return {"truth":..., "refined":x.tolist(), "nfev":nfev, "cost":cost}
```

代码 A-2 React 反演/绘图副作用（chart 在 React 树中）。

```
async function doInv() {
  setBusy(true); setErr("");
  try {
    const data = await postJSON("/api/invert", getParams());
    setInv(data);
  } catch(e) { setErr(String(e)); }
  finally { setBusy(false); }
}
useEffect(() => { if (inv) drawChart(inv); }, [inv]);
```